7.7 Finally is for cleanup

Our web application from 7.6 is complete. In this lesson, create a helper and temporarily use the same entry script for a CLI demonstration. Afterward, restore the full boundary entry listing from 7.5; all model, controller, and template files stay unchanged.

Give cleanup a real resource

Create `src/Tutorials/Class7/StreamDemo.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use RuntimeException;

class StreamDemo
{
    public function describe(int $calories, array &$events): string
    {
        $stream = fopen('php://memory', 'w+');
        if ($stream === false) {
            throw new RuntimeException('Could not open the temporary stream.');
        }
        $events[] = 'Stream opened';

        try {
            $dessert = new Dessert('Berry crumble', $calories);
            $text = (string) $dessert;
            if (fwrite($stream, $text) !== strlen($text)) {
                throw new RuntimeException('Could not write the complete
description.');
            }
            if (!rewind($stream)) {
                throw new RuntimeException('Could not rewind the stream.');
            }
            $result = stream_get_contents($stream);
            if ($result === false) {
                throw new RuntimeException('Could not read the description.');
            }
            $events[] = 'Return prepared';
            return $result;
        } finally {
            fclose($stream);
            $events[] = is_resource($stream) ? 'Stream still open' : 'Stream
closed';
        }
```

```
    }
}
```

php://memory is a temporary in-memory stream; no upload, database, or persistent file is involved. A stream is a resource, not an ordinary object. Check a failed open before entering the block that assumes it was acquired. Some stream failures warn and return false rather than throwing automatically; these checks turn failure results into explicit exceptions. The controlled in-memory example should open successfully.

The try has a finally without a catch. StreamDemo can release its resource but does not know how its caller should respond to bad calories. The domain exception therefore propagates after cleanup. Passing events by reference lets the caller inspect our temporary teaching trace even when no result is returned. These trace strings are not a logging system.

Compare marker, return, and propagation

Temporarily replace `public/tutorials/class7/index.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Dessert;
use App\Tutorials\Class7\NegativeCaloriesException;
use App\Tutorials\Class7\StreamDemo;

require_once __DIR__ . '/../../../vendor/autoload.php';

foreach ([500, -1] as $calories) {
    echo "Marker attempt: {$calories}", '<br>', PHP_EOL;
    try {
        echo new Dessert('Berry crumble', $calories), '<br>', PHP_EOL;
    } catch (NegativeCaloriesException $e) {
        echo 'Negative value caught', '<br>', PHP_EOL;
    } finally {
        echo 'Finally marker', '<br>', PHP_EOL;
    }
}

$demo = new StreamDemo();
foreach ([500, -1] as $calories) {
    $events = [];
    try {
        $result = $demo->describe($calories, $events);
        $events[] = 'Caller received: ' . $result;
    } catch (NegativeCaloriesException $e) {
        $events[] = 'Caller caught negative calories';
    }
    echo implode('<br>'.PHP_EOL, $events), '<br>'.PHP_EOL;
}
```

Run from the repository root:

```
php public/tutorials/class7/index.php
```

The marker appears after the success description and after the caught negative attempt. It is an execution marker, not a statement that the operation succeeded.

The stream demonstrations should end with these two groups:

```
Stream opened
Return prepared
Stream closed
Caller received: Dessert: Berry crumble, 500 calories
Stream opened
Stream closed
Caller caught negative calories
```

On success, the return expression is evaluated, finally closes the stream, and only then does the caller receive the result. On failure, there is no Return prepared or Caller received line. The stream closes before the caller's catch runs. The throw does not resume the skipped write or read statements.

Cleanup does not undo work

Closing a stream releases a resource. If earlier code had already written a persistent file or sent a message, closing its resource would not reverse that action. Similarly, an exception from a later model setter does not roll back earlier successful setters. Our temporary stream disappears when closed; that is a feature of this particular resource, not an automatic rollback policy.

Avoid return in finally: it can replace a pending return or suppress a propagating exception. Avoid throwing during cleanup: a new exception can become the propagated failure and obscure the original problem. Also do not put a success message in finally; it runs on both outcomes.

Finally is part of normal completion and exception unwinding. Do not promise that it runs after exit, a killed process, or every engine failure. We place no exit inside this demonstration.

Restore and verify the web application

Restore `public/tutorials/class7/index.php` using the complete boundary listing from 7.5. Open the planner and verify valid, negative, and excessive values still produce 200, 422, and 422. StreamDemo may remain in src/; it is a teaching helper and is not called by the planner. Neither Application nor the templates need changing.